

SCA 五步：信息源变换全图解

****对应代码****: [models/spatial_attn.py](file:///c:/Users/PC/Documents/trae_projects/bev/models/spatial_attn.py) 的 ``SpatialCrossAttention.forward()`` (第 177-274 行)

> ****核心问题****: 2500 个 BEV Query 是怎么从 6 张图像特征里"取出"属于自己的信息的?

> ****一句话主线****: Query 的空间位置 (3D) → 翻译成图像像素位置 (2D) → 在小数位置取特征 → 加权汇总回 Query。

〇、先记住所有符号的含义（贯穿全文）

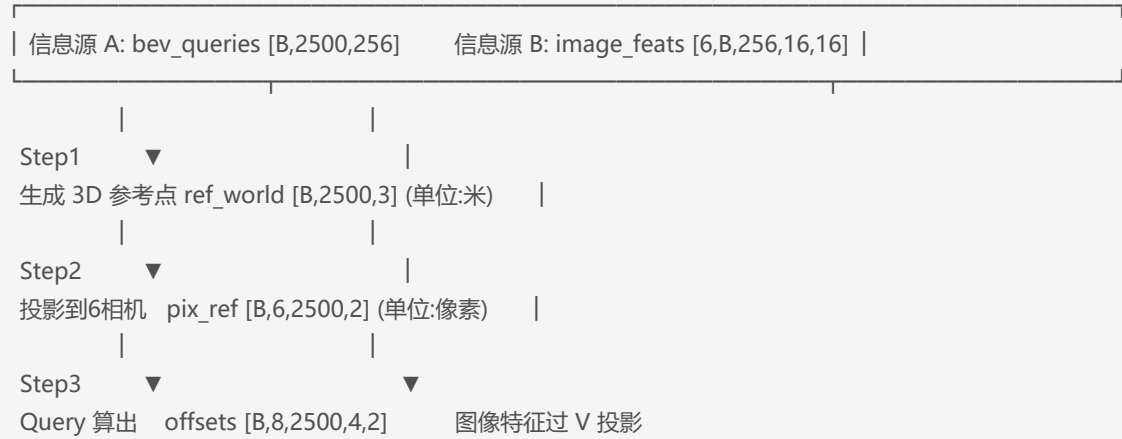
符号	含义	本项目取值
B	batch size（一次处理几个样本）	2
N	BEV Query 数量 = $H \times W \times Z$	$50 \times 50 \times 1 = 2500$
C	特征通道数（Query 的"信息宽度"）	256
num_cam	相机数量	6
$H_f \times W_f$	每张图像特征图大小	16×16
H	注意力头数	8
P	每个 Query 在每个相机采样几个点	4
d	每个头的通道数 = C/H	$256/8 = 32$

【两条信息源（贯穿五步）：】

信息源 A: BEV Query（提问者） $[B, N, C] = [2, 2500, 256]$
信息源 B: 6 张图像特征（信息库） $[6, B, C, H_f, W_f] = [6, 2, 256, 16, 16]$

SCA 做的事 = 让 A 按照几何规则，去 B 里精准取信息，最后还是返回 $[B, N, C]$

一、总览图（一张图看完五步）



weights [B,8,2500,4] feat_v [B,256,16,16]

Step4 像素位置+偏移 → 双线性采样 v_sampled [B,8,2500,4,32]

Step5 weights 加权求和 → 合并头 → 线性层

output [B,2500,256] ← 形状和输入 A 完全一样，但信息已更新！

【注意开头结尾】：输入 [B,2500,256]，输出还是 [B,2500,256]——形状没变，但每个 Query 已经“看过”图像，内容被更新了。这就是 Attention 的本质：【不改形状，只改内容。】

二、Step 1: 生成 3D reference point

信息变换：无 → 网格坐标（归一化）→ 物理坐标（米）

BEV 平面 50×50 网格

| Q0 | Q1 | .. | ... | | 每个格子中心放一个点
| .. | | | | | 共 2500 个点

▼ get_reference_points_in_bev()
ref_norm [1, 2500, 3] 坐标值 ∈ [0,1] (归一化, 均匀切网格)
| expand 到 batch
▼
[B, 2500, 3] = [2,2500,3]
| denormalize: x_米 = x_norm×100 - 50
▼
ref_world [B, 2500, 3] 坐标值 ∈ [-50,50] 米 (真实物理坐标)
| 补一列 1 变成齐次坐标
▼
ref_world_h [B, 2500, 4] = [2,2500,4]

对应代码（第 210-213 行）

```
ref_norm = get_reference_points_in_bev(bev_h, bev_w, bev_z, device) # [1,N,3]
ref_norm = ref_norm.expand(B, -1, -1) # [B,N,3]
ref_world = denormalize_bev_to_world(ref_norm, x_bound, y_bound, z_bound) # [B,N,3]
ref_world_h = torch.cat([ref_world, torch.ones(B, N, 1)], dim=-1) # [B,N,4]
```

这一步信息源发生了什么

阶段	张量	单位	作用
----	----	----	----

归一化	[1,2500,3]	无 [0,1]	均匀切网格，与物理范围解耦
反归一化	[B,2500,3]	米	相机矩阵只认"米"
齐次化	[B,2500,4]	米	补 1 才能和 4×4 外参矩阵相乘

【关键】：这一步只用到网格配置（H/W/bound），不依赖图像。2500 个点是固定的，每个 Query 对应一个。

三、Step 2：3D 点投影到 6 个相机

信息变换：3D 物理坐标 [B,2500,3] → 2D 像素坐标 [B,6,2500,2]

这是信息源从"BEV 空间"跨进"图像空间"的唯一桥梁。【维度上多了一个 num_cam=6】——每个 3D 点要分别问 6 个相机。

```

ref_world_h [B,2500,4] (3D, 米)
|
| 对每个相机 c 做两次变换:
| ① 外参 lidar2cam[c] (4×4): 自车坐标 → 相机坐标
|   P_cam = R·P_world + t
| ② 内参 K[c] (3×3): 相机坐标 → 像素 (透视除法)
|   u = fx·Xc/Zc + cx
|   v = fy·Yc/Zc + cy
|
▼
pix_ref [B, 6, 2500, 2] ← 每个点在每个相机的像素 (u,v)
in_front [B, 6, 2500]   ← 该点是否在相机前方 (z_cam>0)，布尔掩码
    
```

对应代码（第 218-220 行）

```

pix_ref, in_front = project_world_to_image(ref_world_h, lidar2cam, cam_intrinsic)
# pix_ref: [B, num_cam, N, 2] = [2,6,2500,2]
# in_front: [B, num_cam, N]   = [2,6,2500]
    
```

6 个相机发生了什么（掩码的作用）

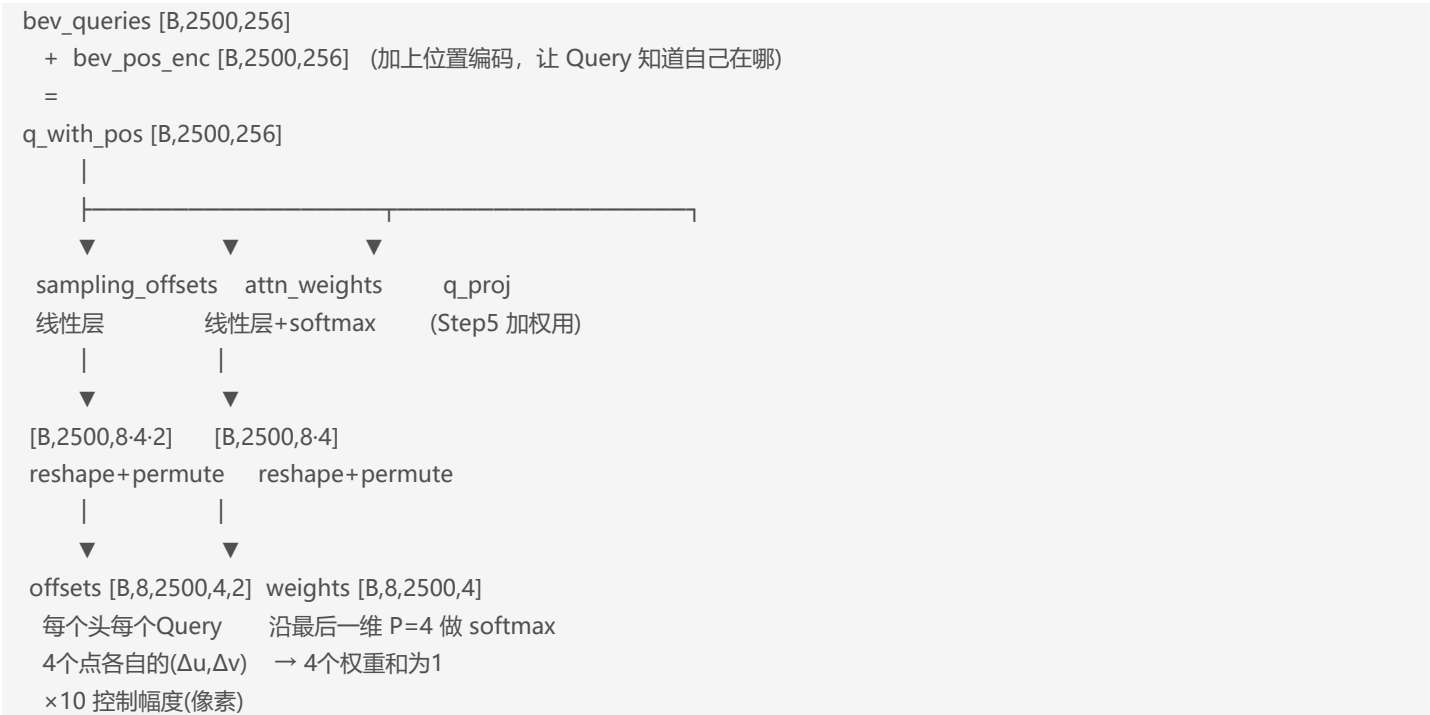


【关键】：投影后一个点通常只在 1-2 个相机里可见，其余相机靠 in_front 掩码在 Step 4 被丢弃。这就是 SCA 天然"只看必要相机"的原因。

四、Step 3: Query 算出采样偏移和注意力权重

信息变换：Query 内容 [B,2500,256] → offsets [B,8,2500,4,2] + weights [B,8,2500,4]

这一步【信息源 A (Query) 开始发挥主观能动性】：投影点是"死"的几何位置，Query 要在它周围学出"精确看哪几个点"和"各信几分"。



对应代码 (第 225-237 行)

```

q_with_pos = bev_queries + bev_pos_enc
offsets = self.sampling_offsets(q_with_pos).view(B,N,H,P,2).permute(0,2,1,3,4) # [B,H,N,P,2]
offsets = offsets * 10.0 # 偏移幅度 (像素)
weights = self.attn_weights(q_with_pos).view(B,N,H,P).permute(0,2,1,3) # [B,H,N,P]
weights = F.softmax(weights, dim=-1) # P 维度归一化
  
```

offsets 和 weights 各自的含义

```

投影点 × (Step2 给的固定位置)
|
| + offsets (学出来的 4 个微调位移)
▼
• • • • ← 实际采样的 4 个点 (可以跑到车的四个角)
w1 w2 w3 w4 ← weights: 4 个点各自的重要程度, 和为 1
  
```

张量	形状	学什么	初始值
offsets	[B,8,2500,4,2]	4 个点在像素平面往哪偏	初始化为 0 (先靠几何站稳)
weights	[B,8,2500,4]	4 个点各信几分	softmax 后均匀 0.25

五、Step 4: 多相机多采样点双线性采样（信息源 A 和 B 在这里汇合）

信息变换：图像特征 [B,256,16,16] + 像素位置 [B,8,2500,4,2] → 采样特征 [B,8,2500,4,32]

【这是五步里唯一让两条信息源真正接触的一步。】遍历 6 个相机，每个相机都采样后累加。

单个相机内部的变换

信息源 B 的一个相机:

feat [B,256,16,16]

| v_proj 线性投影

▼

feat_v [B,256,16,16]

位置 (Step2 + Step3) :

pix_ref[:,cam] [B,2500,2]

投影点

+ offsets [B,8,2500,4,2]

偏移 (广播相加)

▼

pix_sample [B,8,2500,4,2]

最终采样的小数像素坐标

× in_front 掩码 (相机后方的点坐标设成 -1e4, 采到 0)

|

| F.grid_sample(mode="bilinear") ← 双线性采样, 可微

▼

v_sampled [B,256,8,2500,4] reshape/permute

▼

v_sampled [B,8,2500,4,32] = [B,H,N,P,d]

每个头每个Query每个点, 取回32维特征

6 个相机累加 (第 245-265 行的 for 循环)

sampled_sum 初始化为全 0 [B,8,2500,4,32]

cam 0 采样 —|

cam 1 采样 —|

cam 2 采样 —|— sampled_sum += v_sampled (6 个相机的特征直接相加)

cam 3 采样 —| 看不到该点的相机贡献 0 (被 mask)

cam 4 采样 —|

cam 5 采样 —|

▼

sampled_sum [B,8,2500,4,32]

对应代码 (第 245-265 行核心)

for cam_idx in range(num_cam):

feat_v = self.v_proj(image_feats[cam_idx]) # [B,C,Hf,Wf]

pix_sample = pix_ref[:,cam_idx].unsqueeze(1).unsqueeze(3) + offsets # [B,H,N,P,2]

pix_sample = pix_sample * mask + (1-mask) * (-1e4) # 屏蔽相机后方

v_sampled = self.bilinear_sampling(feat_v, pix_sample) # [B,C,H,N,P]

v_sampled = v_sampled.view(B,H,d,N,P).permute(0,1,3,4,2) # [B,H,N,P,d]

sampled_sum = sampled_sum + v_sampled # 跨相机累加

为什么必须双线性（再强调）

采样坐标是小数：(180.3, 120.7)
 → 取周围 4 个整数邻居 (180,120)(181,120)(180,121)(181,121)
 → 按距离加权平均
 两个作用：① 任意亚像素位置都能取 ② 采样操作可微，梯度能传回 offsets

【信息源变换的本质】：图像本来是"16×16 的空间网格"，经过这一步被重新组织成"2500 个 Query × 4 个点"的特征——信息从"按像素排列"变成了"按 Query 排列"。

六、Step 5：注意力加权求和 + 输出投影

信息变换：[B,8,2500,4,32] → [B,2500,256]，形状回到起点

```
sampled_sum [B,8,2500,4,32]  6相机累加来的特征
weights [B,8,2500,4,1]  4个点的权重(softmax,和为1)
| 逐元素相乘后沿 P=4 这一维求和
▼
output [B,8,2500,32]  4个采样点加权融合成1个
| permute + 合并 8 个头: 8×32 = 256
▼
output [B,2500,256]
| out_proj 线性层
▼
output [B,2500,256] ★ 和输入 bev_queries 形状完全一致
```

对应代码（第 271-274 行）

```
output = (weights.unsqueeze(-1) * sampled_sum).sum(dim=3) # [B,H,N,d] 沿P求和
output = output.permute(0,2,1,3).contiguous().view(B,N,C) # 合并头 [B,N,C]
return self.out_proj(output) # [B,N,C]
```

这一步信息源发生了什么

操作	张量变化	含义
沿 P 加权求和	[B,8,2500,4,32]→[B,8,2500,32]	4 个采样点按重要性融合成 Query 的新特征
合并头	[B,8,2500,32]→[B,2500,256]	8 个头看到的不同信息拼回来
out_proj	[B,2500,256]→[B,2500,256]	可学习的输出变换

七、完整形状追踪表（背诵版）

步骤	关键张量	形状（用真实数字）	所在空间
输入A	bev_queries	[2, 2500, 256]	BEV Query

输入B	image_feats	[6, 2, 256, 16, 16]	6 张图像
Step1	ref_world_h	[2, 2500, 4]	3D 物理 (米)
Step2	pix_ref	[2, 6, 2500, 2]	2D 像素
Step2	in_front	[2, 6, 2500]	可见性掩码
Step3	offsets	[2, 8, 2500, 4, 2]	像素偏移
Step3	weights	[2, 8, 2500, 4]	采样权重
Step4	pix_sample	[2, 8, 2500, 4, 2]	最终采样位置
Step4	v_sampled	[2, 8, 2500, 4, 32]	取回的特征
Step4	sampled_sum	[2, 8, 2500, 4, 32]	6相机累加
Step5	output (加权后)	[2, 8, 2500, 32]	沿P融合
Step5	output (合并头)	[2, 2500, 256]	回到 BEV Query

八、信息源变换的三个"跳跃" (理解了就真懂了)

跳跃 1 (Step 1→2) : 坐标系跳跃

3D 米 [B,2500,3] —投影— 2D 像素 [B,6,2500,2]

维度多了 num_cam=6: 一个空间点要分别问 6 个相机

跳跃 2 (Step 3) : 静态→动态跳跃

投影点是固定几何, 加上 Query 自己学的 offsets/weights

变成"会主动寻找信息"的采样位置

跳跃 3 (Step 4) : 信息源汇合跳跃

图像特征按"像素网格"组织 —采样— 按"Query"重新组织

[B,C,16,16] → [B,H,N,P,d]

这是两条信息源唯一接触的地方

【最后记住】:

- Query 的形状 [B,N,C] 五步里**进什么样、出什么样**, 变的只是内容
- 真正搬运信息的是 Step 2 (几何寻址) + Step 4 (可微采样)
- Step 3 提供"学习能力", Step 5 负责"汇总"
- 这就是"几何指导的稀疏注意力": 用相机几何把全图注意力 (1 亿次) 压缩到只采 6×4 个点 (6 万次)

建议配合运行 demo 看 frame0_projection.png: 红点就是 Step2 的 pix_ref, Step3 的 offsets
训练后会让红点微微散开, Step4 就在这些位置取特征。